

Question 1: Employee Payroll System

Create a base class `Employee` with:

- `employee_id`
- `name`
- `base_salary`

Create the following child classes:

- `FullTimeEmployee`
- `PartTimeEmployee`
- `ContractEmployee`

Implement a `calculate_salary()` method in each class.

Rules:

- Full-time employees receive base salary.
- Part-time employees are paid based on hourly rate and hours worked.
- Contract employees receive a fixed contract amount.

Use polymorphism to calculate salaries for different employees.

Question 2: Shape Hierarchy

Create an abstract class or base class called `Shape`.

Create the following subclasses:

- `Circle`
- `Rectangle`
- `Triangle`

Each class should implement:

- `area()`
- `perimeter()`

Create a list containing different shapes and use polymorphism to display the area and perimeter of every shape.

Question 3: Bank Account Hierarchy

Create a base class `BankAccount`.

Create the following subclasses:

- `SavingsAccount`
- `CurrentAccount`

Implement:

- `deposit()`
- `withdraw()`
- `display_balance()`

Additional rules:

SavingsAccount:

- Cannot withdraw more than the balance.
- Provide an `add_interest()` method.

CurrentAccount:

- Allow overdraft up to a specified limit.

Use encapsulation to protect the account balance.

Question 4: University Management System

Design a university management system using OOP.

Create classes:

- `Person`
- `Student`
- `Professor`
- `Course`
- `Department`

Requirements:

- A student can enroll in multiple courses.
- A professor can teach multiple courses.
- A course can have multiple students.
- A department can contain multiple professors and students.

Implement methods to:

- Enroll students.
- Assign professors.
- Display course information.
- Display student courses.
- Display professor courses.

Use composition or aggregation where appropriate.

Question 5: Vehicle Rental System

Create a vehicle rental application.

Create a base class:

`Vehicle`

Create subclasses:

- `Car`
- `Bike`
- `Truck`

Each vehicle should have:

- Registration number
- Brand
- Model
- Rental price per day
- Availability status

Implement:

- `rent()`
- `return_vehicle()`
- `calculate_rent(days)`

Different vehicle types should calculate rental charges differently.

Create a `RentalSystem` class to manage all vehicles.

Question 6: E-Commerce Order System

Create the following classes:

- `Product`
- `Customer`
- `Cart`
- `Order`
- `Payment`

Requirements:

- Customer can add products to a cart.
- Customer can remove products.
- Cart calculates subtotal.
- Order calculates taxes and discounts.
- Payment processes the final amount.

Implement different payment classes:

- `CreditCardPayment`
- `UPIPayment`
- `CashOnDelivery`

Use polymorphism for payment processing.

Question 7: Hospital Management System

Design a hospital management system using OOP.

Create classes:

- Person
- Patient
- Doctor
- Appointment
- Prescription

Requirements:

- A patient can have multiple appointments.
- A doctor can have multiple appointments.
- An appointment connects a doctor and a patient.
- A doctor can issue prescriptions.
- A patient can have multiple prescriptions.

Implement methods to:

- Schedule appointments.
 - Cancel appointments.
 - Display patient history.
 - Display doctor schedule.
 - Create prescriptions.
-

Question 8: Library Management System

Build a library management system using OOP.

Create classes:

- Book
- Member
- Library
- Transaction

Requirements:

- Library maintains a collection of books.
- Members can borrow books.
- Members can return books.
- A book cannot be issued to two members simultaneously.
- Track borrowing and returning dates.
- Calculate a fine for overdue books.

Use encapsulation for book availability and member borrowing information.

Question 9: Notification System

Create a notification framework.

Create a base class:

Notification

Create subclasses:

- **EmailNotification**
- **SMSNotification**
- **PushNotification**

Each class should implement:

```
send(message, recipient)
```

Question 10: Food Delivery Application

Design a simplified food delivery application.

Create classes:

- Customer
- Restaurant
- MenuItem
- Order
- DeliveryAgent
- Payment

Requirements:

- Restaurants can have multiple menu items.
- Customers can place orders.
- Orders can contain multiple menu items.
- Customers can make payments.
- Delivery agents can be assigned to orders.
- Order status should support:
 - PLACED
 - CONFIRMED
 - PREPARING

- OUT_FOR_DELIVERY
- DELIVERED
- CANCELLED

Implement the complete order lifecycle using OOP.